University of Antwerp
Faculty of Science

Programming Project Databases

Assignment 2025–2026

University of Antwerp

Web-based Club Ladder

Professor:
Brent van Bladel

Stakeholder:
Tim Apers

Supervisors:
Sam Pieters

Scrum Master:
Jochem Turelinckx

February 17, 2024

1 Introduction

In the course *Programming Project Databases*, you will learn to develop an extensive software project in a team setting. This assignment is designed with the following learning objectives in mind:

- Build and use a database inside a software product.
- Work independently in a team.
- Find and learn about new technologies on your own.
- Plan work and distribute tasks.
- Think creatively and solve problems.
- Clearly report on progress and choices made.
- Write high-quality software with a focus on usability, efficiency, and extensibility.
- Deploy and maintain a software system in production.

This is a comprehensive project course. There is **no** written exam in June, and there is **no** retake in August. Your evaluation will be based on the delivered software product, taking into account the above objectives.

In addition to this introduction, you will find section 2 explains the basic requirements of this year's assignment, and section 3 where all practical matters and the evaluation process are explained in this document.

2 Assignment

The assignment is to create a web-based club ladder application for local sports clubs (specifically tennis and padel) where members can register, join an internal ranking ("ladder"), get periodically assigned matches, and report results. A club ladder is a flexible, ongoing competition in which participants move up or down the ranking based on match outcomes. An explanation and implementation of how a ladder competition works can be found here. In essence, players challenge opponents, schedule matches, and gain or lose points depending on the level of their opponents. The system must support both individual sports (tennis) and team-based sports (padel teams of two), and must be implemented in a way that enables the addition of new sports with minimal changes.

The assignment is intentionally open-ended: each team is expected to give their own interpretation, propose additional features, or refine match-scheduling and rating systems. Beyond the required functionality listed in this document, teams are free to innovate.

Each team has to submit a scope description by the end of the month (see section 3.6: Planning) that contains their plans for the application.

In the sections 2.1 and 2.2 more information about the basic requirements of the application in terms of entities and functionality is described. section 2.3 provides a list of technologies

that can be used.

2.1 Entities

The following entities must be present in your application. You may add fields or extend relationships as needed.

User A person using the application, including players, team members, and club administrators. Users can log in, join clubs, join ladders (individually or as teams), report match results, and receive notifications.

Club A sports club that contains users and one or more ladders. Clubs have administrators who manage members, ladders, and club-specific rules.

Ladder (Ranking)
An internal ranking for a specific sport within a club. A ladder holds participants, their current ranking or points, configuration rules (such as challenge limits and match frequency), and any sport-specific settings.

Team
An entity registered in a ladder, either a single user for tennis or a team of two users for padel. Each team includes availability settings as well as a numerical rating or point total.

Match
A scheduled or completed contest between two teams, either two individuals for tennis or two teams for padel. Each match contains a scheduled time, status, and the eventual result.

2.2 Basic Requirements

This section describes the minimum required functionality that all teams must implement. All functionality must be demonstrated during the final evaluation (see section 3.6: Planning) in order to pass. Teams are free to expand beyond these requirements.

Authentication
Users must be able to register and log in using a username and password. The system should support additional roles, such as club administrators, which allow access to sensitive functionality. While advanced security features are optional, they can earn extra points if they are properly implemented and documented.

Club & Ladder Management
Users can join clubs and participate in their activities. Club administrators should have the ability to create, edit, and delete ladders within their clubs. Each ladder can be configured with specific challenge rules, scheduling frequency, rating parameters, and the sport type, either tennis or padel.

Team Management
Users can join a tennis ladder individually or create or join a two-person team for padel ladders. Teams must be able to define their availability for match scheduling,

and they should appear in the ladder with a rating or points value that reflects their performance.

Ranking & Rating System
: Each ladder must maintain a numerical rating or point total for each team. You are responsible for designing and implementing your own ELO-like or alternative rating formula. The choice of formula, calculation method, and any additional factors is flexible, provided it is clearly explained and justified in the accompanying report. Rating updates must reflect match outcomes and, where applicable, differences in skill or previous performance between players or teams. The system should reliably handle both individual participants (tennis) and two-player teams (padel), and you are encouraged to explore different approaches or optimizations, as long as the implementation remains consistent and produces sensible, reproducible results.

Scheduling & Match Assignment
: The system must periodically assign matches between teams according to ladder rules. Scheduling should consider team availability, sport type (individual or team), challenge limits (for example, allowing challenges only within a certain number of rungs), and fairness to avoid repeated match-ups. The scheduler may operate automatically, semi-automatically, or as a hybrid system, but it must be fully functional. You are expected to explain and justify their scheduling algorithm in the report.

Match Lifecycle & Result Reporting
: Teams must be able to view scheduled matches and confirm or decline match proposals. After completing a match, teams should report the results, which will update the ladder ranking according to the implemented rating system. Including a basic confirmation or dispute mechanism is recommended to ensure accuracy, although it is not mandatory.

External API
: Additionally, you are encouraged to explore integration with external APIs to enrich the system or support scheduling and match planning. For example, you could fetch weather forecasts for the planned match day, retrieve local court availability data, or connect to external sports ranking systems. The specific API choice, the type of data used, and how it is incorporated into the system are fully flexible, allowing you to experiment with different sources, formats, or creative enhancements while documenting your implementation choices.

Beyond these requirements, you must ensure a user-friendly web interface that is both intuitive and appealing. You can use a CSS library, as explained in section 2.3: Technology. Additionally, it is necessary to utilize an **A**pplication **P**rogramming **I**nterface (API) to implement some of the features. The API should adhere to the REST design principle, which includes, among other things, not maintaining state in the API and focusing on entities rather than operations.

The API provides a way to communicate directly with the server from the front end without having to refresh or reload the page, which can significantly enhance user-friendliness. In summary, carefully consider how you implement the features. Well-thought-out choices and designs can prevent a lot of unnecessary work. Always thoroughly justify these choices in your reports.

2.3 Technology

Inherently to this project, you will have to work with various new technologies. There is a wide range of tools, frameworks, libraries, and services that you must combine to implement the assignment. We suggest these "technologies" per category.

Do not let the choice of which technologies to use linger for too long. We recommend comparing some options as soon as possible and discussing with your team which ones you will proceed with. After the main choices are made, you can start reading documentation and tutorials to delve into the chosen technologies.

Webserver

- Flask https://flask.palletsprojects.com/
  The Flask framework in Python is recommended, but using a different programming language and/or webserver is also allowed.

Web Design

- HTML + CSS + Js
  These are the basic elements of every webpage.
- Front end framework
  Modern web pages are often written with a lot of logic on the client and clever use of asynchronous calls to create so-called "single-page applications". These frameworks support this design:
  - Vue.js https://vuejs.org/
  - React https://reactjs.org/
  - Angular https://angular.io/
- CSS Framework
  Using existing code for CSS can be useful, especially for mobile-first development. Here are some examples of popular CSS frameworks:
  - Bulma https://bulma.io/
  - Bootstrap https://getbootstrap.com/
  - Material https://material.io/
- Data Visualization
  To visualize data, such as making plots, there are also handy libraries available online, including:
  - Google Charts https://developers.google.com/chart
  - Chart.js https://www.chartjs.org/
  - d3js https://d3js.org/

Database

- PostgreSQL
  The use of PostgreSQL is mandatory.

- Redis https://redis.io/
  Redis is an in-memory data store that can be used for caching values that need to be updated often.

Database Design

- DBdiagram https://dbdiagram.io/
  Online tool for drawing ER diagrams.
- DBdesigner https://www.dbdesigner.net/
  Same.

API

- JSON Web Tokens https://jwt.io/
  Standard for token-based claims. Can be used for stateless authentication.
- Apiary https://apiary.io/
  Online tool for documenting (and testing) APIs.

Teamwork & Planning

- Git
  The use of a version control system is mandatory. These services often also offer, among other things, *projects* and *issues* for planning and organization. Once a repository is created, give `Sam Pieters` read access. Choose from the following three services:
  - GitHub https://github.com/
  - Bitbucket https://bitbucket.org/
  - Gitlab https://gitlab.com/
- Miro https://miro.com/
  Miro is an "all-in-one workspace"that can be used for planning, organizing, and structuring. A handy tool for discussing and developing ideas.
- Jira https://www.atlassian.com/software/jira
  Agile development tool with support for backlog, sprints, roadmaps, and kanban boards. The use of Jira is mandatory (with a free license).

3 Practical

3.1 Teams

This project should be carried out in teams of 5 or 6 students. The teams were randomly assigned in advance and can be found on Blackboard.

3.2 Sprints

To manage the project, we adhere to the principles of Agile software development. A detailed introduction to this will be provided during the first class. Subsequently, your first *sprint*

will begin. A sprint is a period of 2–3 weeks during which you work towards a specific goal. Jochem Turelinckx will guide you, and at the end/beginning of each sprint, they will conduct a *scrum meeting* with your team.

Since a maximum of 4 meetings can be held per week, we have divided the teams into two groups: A and B, spread across the weeks. Teams with an odd number are in group A, and teams with an even number are in group B:

Group A: Teams 1, 3, 5, and 7
Group B: Teams 2, 4, 6, and 8

Refer to section 3.6: Planning for the schedule of when you are expected. In-person attendance at these meetings is mandatory for all team members and will be taken into account for the final score.

In addition to the comprehensive meeting at the end of a sprint, it is also advisable to meet in between to discuss things and collaborate. We recommend scheduling when you want to work on the project and starting those times with a short meeting called *daily* or *stand up*.

A useful tool to keep track and apply agile principles is Jira. Each team must create a project here and invite us so that we can closely follow your progress. Add the following users:

- jochem.turelinckx@m2q.be
- brent.vanbladel@uantwerpen.be

When aligning the goals of the sprint, also consider the *Definition of Done (DOD)*. This is a checklist of conditions that all features must meet to be considered fully finished. Decide together what the DOD entails, and it must include at least the following:

- Code works (no bugs).
- Code is tested (automatic tests or manual with documented step-by-step scenarios).
- Documentation is completed (not just comments in code, but also a report).
- Feature has been reviewed by a team member.

3.3 Template

To help you get started, a template web application is available on Blackboard. Start by getting this code working locally for each team member separately (following the tutorial in the README). Afterward, you can deploy a version together on the production server (see section 3.4: Hosting).

3.4 Hosting

Exclusively for this course, a budget on the Google Cloud Platform is provided through Google's educational program. You can use this amount to rent a server per team for the duration of the course. It is, of course, not allowed to use this credit for personal purposes!

The specifics of how to use the Google Cloud Platform to host a web application will be explained during the practical sessions. To link you to the educational credit, one team member must provide a Gmail email address.

By the end of the first week, the given web application template must be working on the server (at least). This template can be incrementally expanded to perform the assignment. Use a Git repository (GitHub/GitLab/Bitbucket) with the *production* or *main* branch on the server. Furthermore, a DNS name will also be linked to your server in the form of: `team[x].ua-ppdb.me`.

A working version of your system must be running on the server at all times. Therefore, it is essential for each team member to set up a local development environment with a local test database. This allows you to implement new features smoothly and independently without affecting the production version.

Those interested can use Continuous Integration and Continuous Delivery tools such as Jenkins, CircleCI and Github Actions to automate this process.

3.5 Reporting & Presentation

Evaluation is done through three (interim) *reports* and *demos*. Do not underestimate the importance of documenting, and presenting! Functionality that we are not aware of cannot be assessed. Additionally, a cool idea is worthless if not thoroughly explained and justified.

Report

Technical documentation is part of the final product. It includes the design of the program as a whole, a database diagram with explanations, and a description of the functionality. In summary, it contains all technical information needed for the delivered system. Ensure that each team member is involved in the implementation, not just in project management or documentation.

Presentation

For presentations, we expect a working demonstration, where you receive feedback from the jury. A significant portion of the points will be based on the integration of the required functionality. Two interim presentations will be held during the semester, followed by a final presentation during the exam period of an online available web application.

For a demo, use your own laptop(s), so we strongly recommend checking everything thoroughly in advance (internet connection, connection to the projector, etc.) to ensure a smooth demo. Provide sufficient data and prepare a scenario in advance. While the report is for technical information, the demo should focus on functionality. Use the presentation mainly to demonstrate the features you are proud of. Slides are not necessary as long as the demonstration is clear. Make sure everyone participates in the presentations.

3.6 Planning

An overview of all important dates is given in Table 1 (weeks run from Monday to Monday). Unless stated otherwise, the deadline is 23:59.

For the *first milestone*, submit a scope description for your project on Blackboard. This includes a general description of the application along with a list of features that your team

Tabel 1: Overview of planning and deadlines (red†).

Week	Date	Deadline/Planning
1	09/02/2026	Assignment and introduction to Google Cloud Platform
	15/02/2026†	Hello World application running on GCP
2	16/02/2026	Scrum theory, and planning Sprint 1 — teams A+B
	22/02/2026†	Scope Description A+B (milestone 1)
3	23/02/2026	Sprint 2 — teams A
4	02/03/2026	Sprint 2 — teams B
5	09/03/2026	Sprint 3 — teams A
6	16/03/2026	Sprint 3 — teams B
7	23/03/2026	Sprint 4 — teams A
8	30/03/2026	Sprint 4 — teams B
9 10		Easter break
11	20/04/2026	Sprint 5 — teams A (milestone 2)
12	27/04/2026	Sprint 5 — teams B (milestone 2)
13	04/05/2026	Sprint 6 — teams A
14	11/05/2026	Sprint 6 — teams B
	17/05/2026†	Feature Complete — teams A+B
13	18/05/2026	Sprint 6 — teams A+B
16-	Exam	Final report (Blackboard, date depends on exam) Final presentation (See exam schedule for date)

will implement. Make sure to list and explain all your design decisions as well as any other relevant information that has been decided in your team. During the first sprint demo, each team briefly presents their scope description for feedback.

For the *second milestone*, you will be evaluated based on an online available demo of the application. We expect all decisions to be made regarding the design and database. Provide mockups (pages without a backend connection or drawings) for pages that are not yet implemented. This evaluation will take place during the fourth sprint demo.

During the *final presentation*, we expect a live version of the complete finished product, possibly with your own extensions.

3.7 Evaluation

In addition to functionality, you will also be assessed on the following criteria:

- Teamwork and planning. (4)
- Quality of reports and presentations. (2)

- Quality of program code. (4)
- Quality of the database design and SQL queries. (3)
- Quality, originality, and usefulness of the delivered features. (3)
- Quality of the demonstrations. (2)
- User-friendliness of the application. (2)

Although the course is called "Programming Project Databases", the focus is certainly not only on the database aspect but also on programming a large project and being able to work independently in a team.

With a fair distribution of tasks, and if everyone is capable of presenting their part and answering questions, all members of the group will receive the same points.

3.8 Contact

Every Monday from **8:30 to 12:45**, room **M.G.025** is reserved for meetings or working on the project together.

The teaching assistant is present to discuss specific questions and issues related to the project. For urgent questions about the project, personal problems between team members, or technical issues, you can contact him via email: sam.pieters@uantwerpen.be.

Good luck!